INE Hunting with ELK Lab 3

ACTIVITY STATUS: FINISHED Report an issue ✕

Overview Tasks Solutions

Not Running

For the best experience, choose the region closest to you. Then, start the lab to begin.

MARK UNFINISHED

Select the region closest to you

US-East

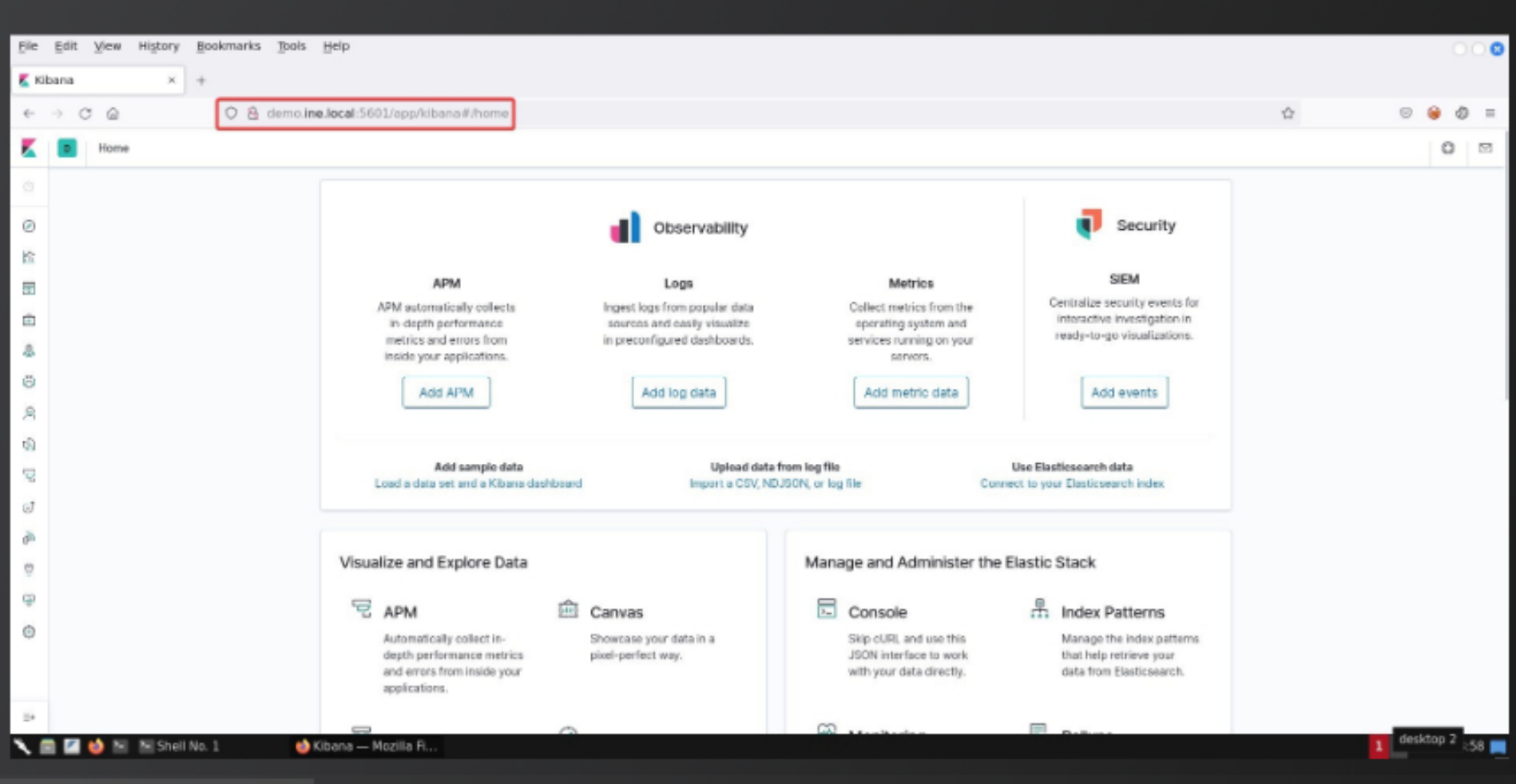
Keyboard layout: US English (US)

Start lab

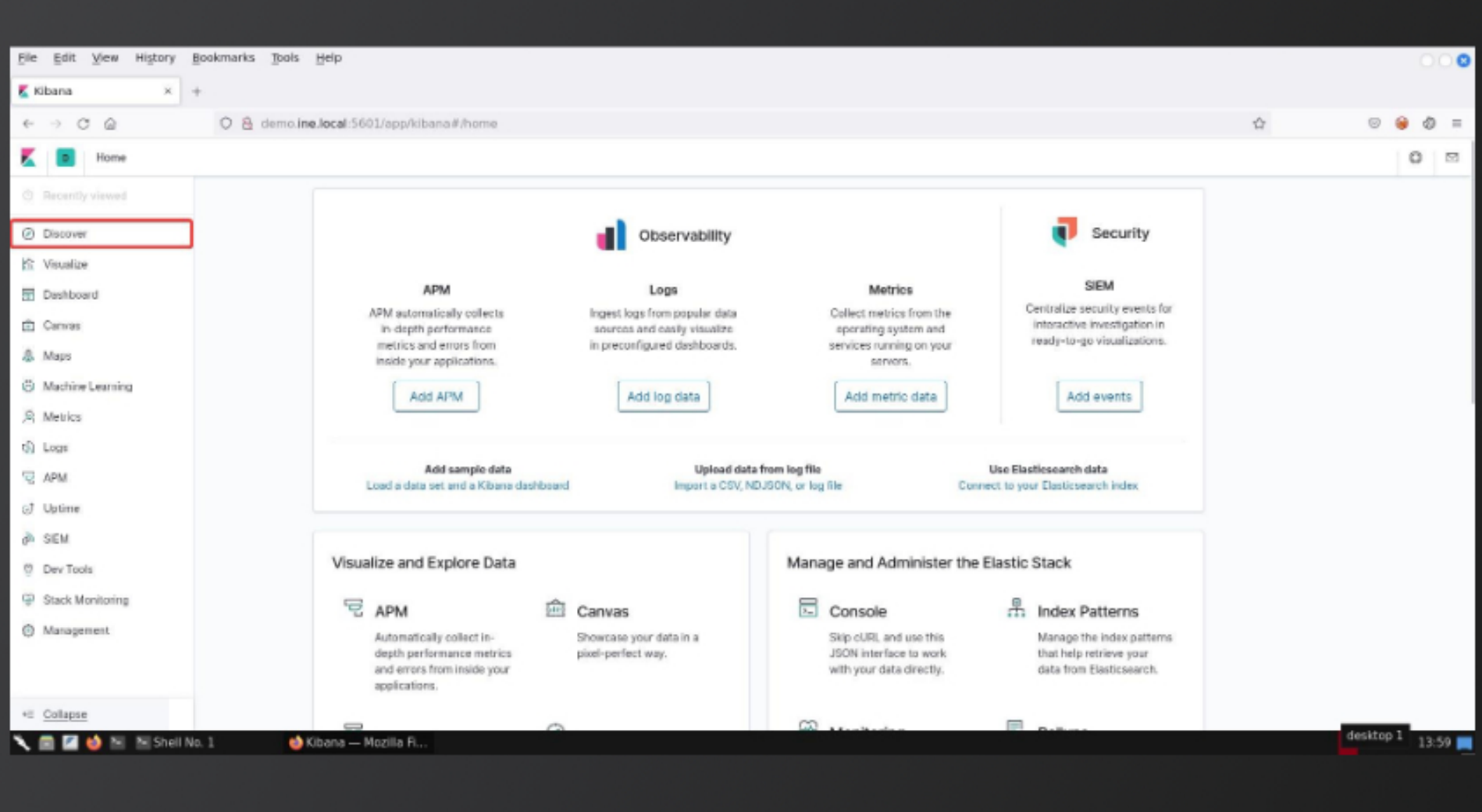
SOLUTIONS

Below, you can find solutions for each task. Remember though, that you can follow your own strategy, which may be different from the one explained in the following lab.

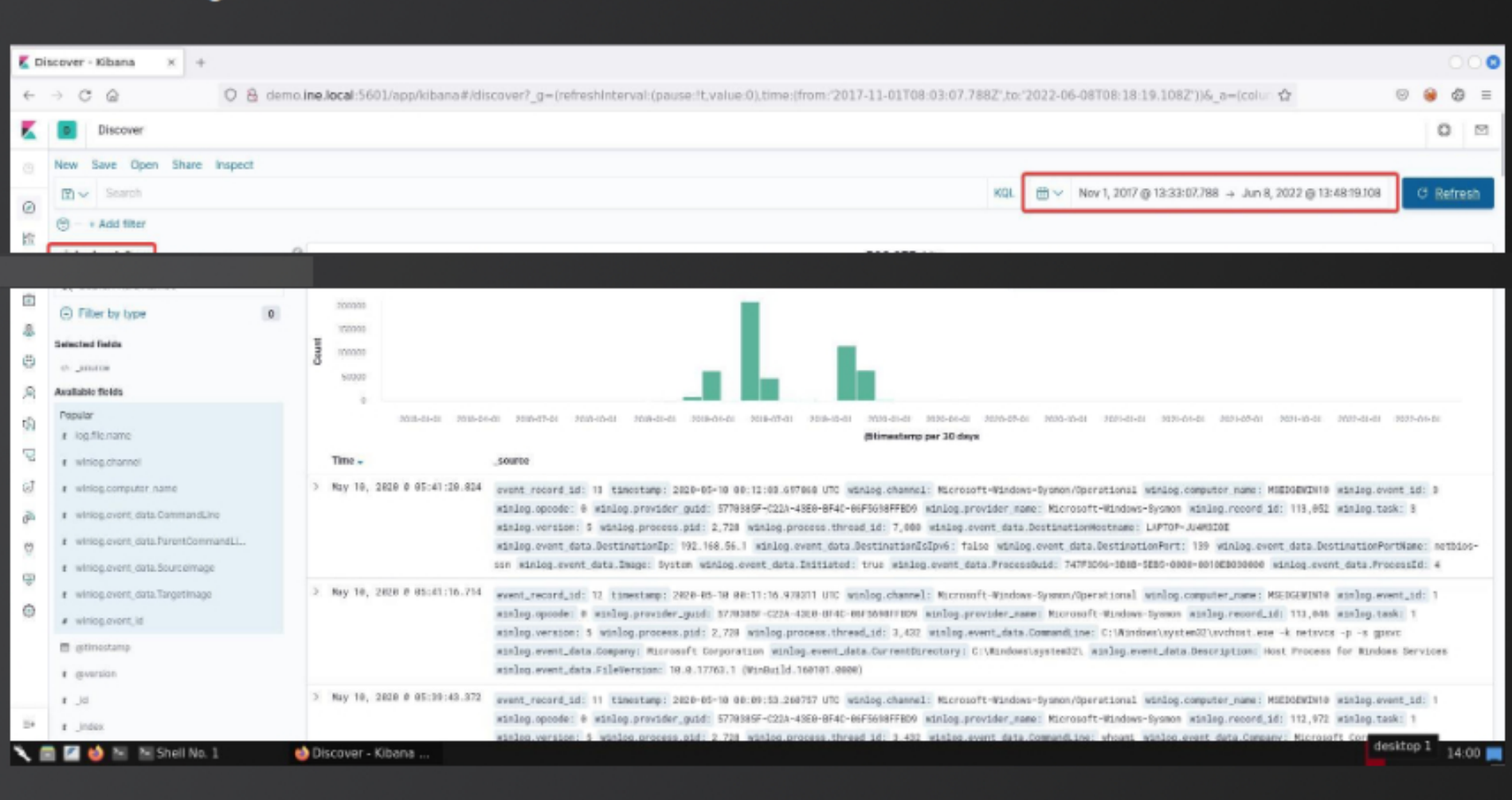
Start **firefox** browser and access Kibana at **http://demo.ine.local:5601**



Go to **Discover**



Select **winlogbeat-*** and select start date from **November-1-2017** to till date.



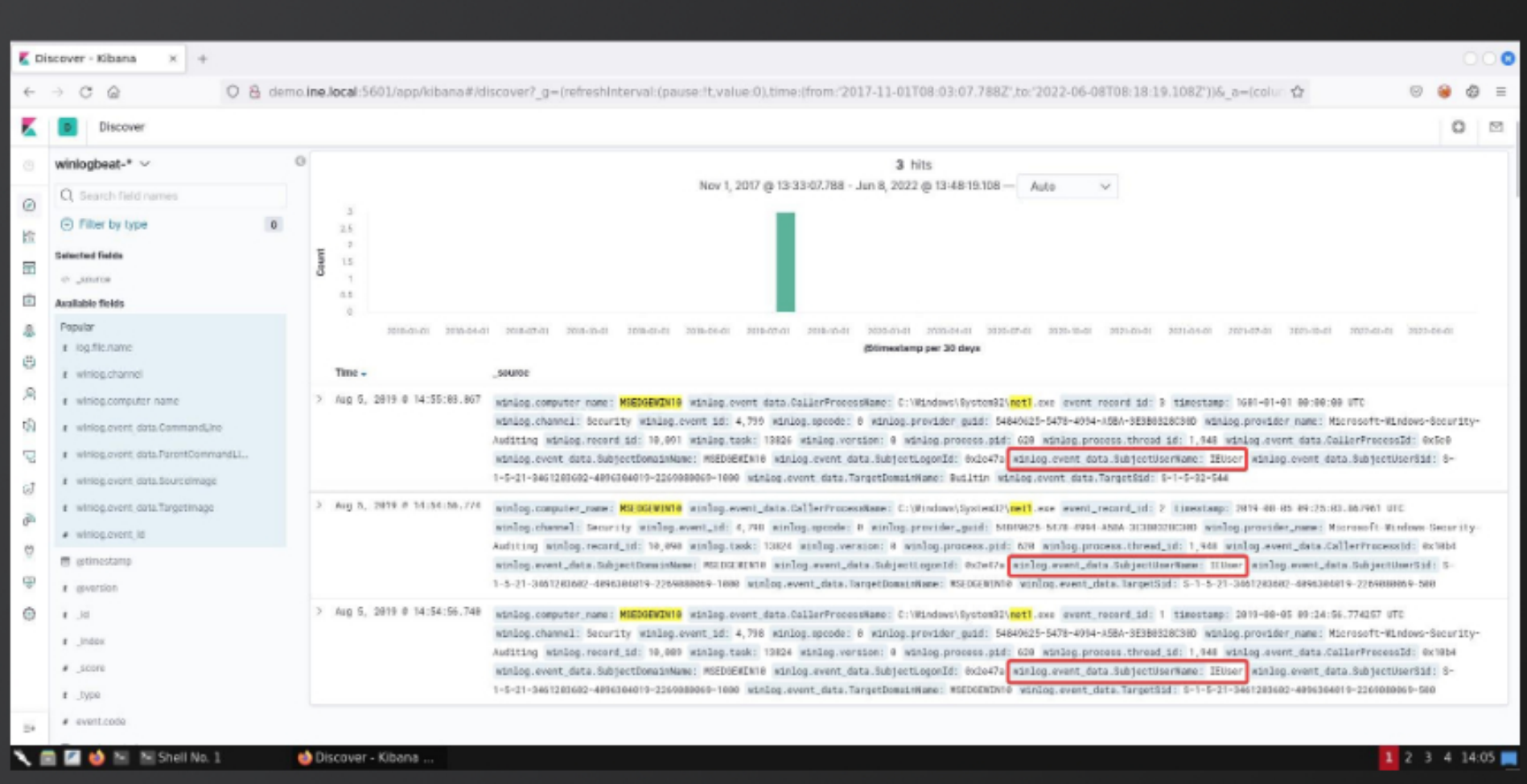
Task 1. Perform a hunt for account discovery

In this task, we begin by reading the documentation in the hint's area. After going through it, we'll discover that "net" is a utility often used for this enumeration. Moreover, Windows 10 has introduced two new events, which are quite interesting:

- 4798 -- A user's local group membership was enumerated
- 4799 -- A security-enabled local group membership was enumerated

```
winlog.event_id:(4798 OR 4799) AND winlog.event_data.CallerProcessName:(net OR net1) AND winl
```

Once we execute the query, we get the following output (for brevity, only display user who did the action and the user/group name that was being enumerated):



The user IEUser has been enumerating the built-in group Administrators, and the built-in account Administrator.

Task 2. Hunt for Persistence through Accessibility Features

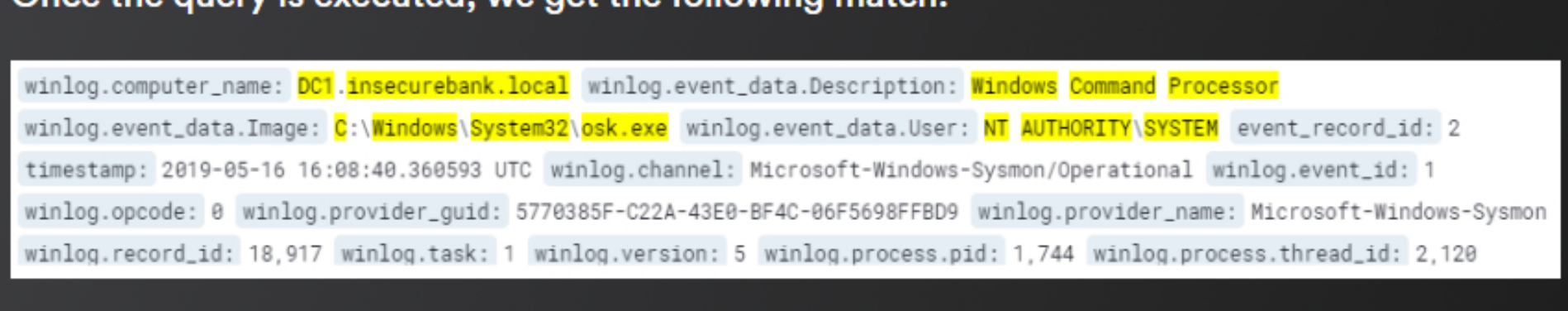
Reviewing the information available in T1015, we assemble the following list of targeted executables:

- Sethc.exe
- Utilman.exe
- Osk.exe
- Magnify.exe
- Narrator.exe
- DisplaySwitch.exe
- AtBroker.exe

Now, taking into account that the hint refers to file replacement with cmd.exe, what we would be interested into, is if a file with any of the names listed above contains the Description field that matches cmd.exe -- essentially, we are looking for renamed cmd.exe. Finally, the user it would execute as is "NT AUTHORITY/SYSTEM". The query we construct is as follows:

```
winlog.event_data.Image:("C:\Windows\System32\osk.exe" OR "C:\Windows\System32\sethc.exe" OR
```

Once the query is executed, we get the following match:



```
winlog.computer_name: DC1.insecurebank.local winlog.event_data.Description: Windows Command Processor
winlog.event_data.Image: C:\Windows\System32\osk.exe winlog.event_data.User: NT AUTHORITY\SYSTEM event_record_id: 2
timestamp: 2019-05-16 16:08:40.360593 UTC winlog.channel: Microsoft-Windows-Sysmon/Operational winlog.event_id: 1
winlog.opcode: 0 winlog.provider_guid: 5770385F-C22A-43E0-BF4C-06F5698FFBD9 winlog.provider_name: Microsoft-Windows-Sysmon
winlog.record_id: 18,917 winlog.task: 1 winlog.version: 5 winlog.process.pid: 1,744 winlog.process.thread_id: 2,120
```

Task 3. Hunt for Privilege Escalation through Scheduled tasks

Reading through the hinted document, we create a list of writable locations by all users, which is suspicious to run tasks from (remember to filter out known-good!). These locations are:

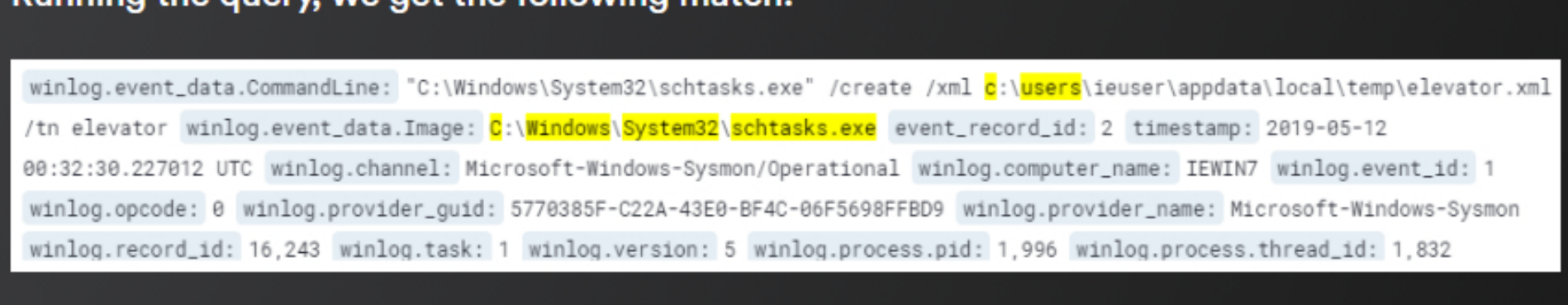
- C:\users\*
- C:\programdata\*
- C:\Windows\Temp\*

Now for this hunt, we have only Sysmon logs available, therefore for our detection capability will focus on Event ID 1 (Process creation) specifically looking for execution of schtasks.exe with a command line parameter that points to one of the locations listed above.

Armed with this knowledge, we create the following query:

```
winlog.event_id:1 AND winlog.event_data.Image:"C:\Windows\system32\schtasks.exe" AND winlog.e
```

Running the query, we get the following match:

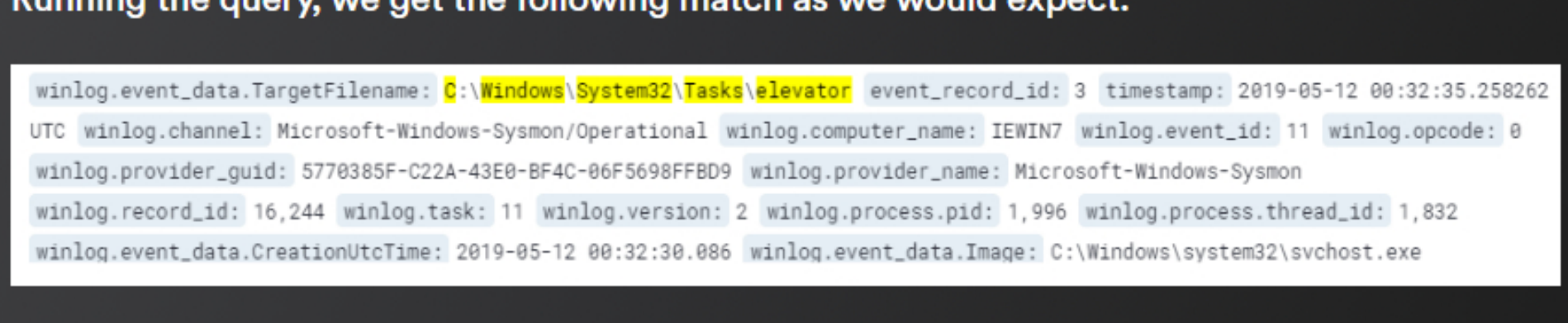


```
winlog.event_data.CommandLine: "C:\Windows\System32\schtasks.exe" /create /xml c:\users\ieuser\appdata\local\temp\elevator.xml
/tn elevator winlog.event_data.Image: C:\Windows\System32\schtasks.exe event_record_id: 2 timestamp: 2019-05-12
00:32:30.227012 UTC winlog.channel: Microsoft-Windows-Sysmon/Operational winlog.computer_name: IEWIN7 winlog.event_id: 1
winlog.opcode: 0 winlog.provider_guid: 5770385F-C22A-43E0-BF4C-06F5698FFBD9 winlog.provider_name: Microsoft-Windows-Sysmon
winlog.record_id: 16,243 winlog.task: 1 winlog.version: 5 winlog.process.pid: 1,996 winlog.process.thread_id: 1,832
```

[illegible] activity, we can leverage Sysmon Event ID 11 (File create) to identify the creation, by querying for:

```
winlog.event_data.TargetFilename:"C:\Windows\System32\Tasks\elevator"
```

Running the query, we get the following match as we would expect:

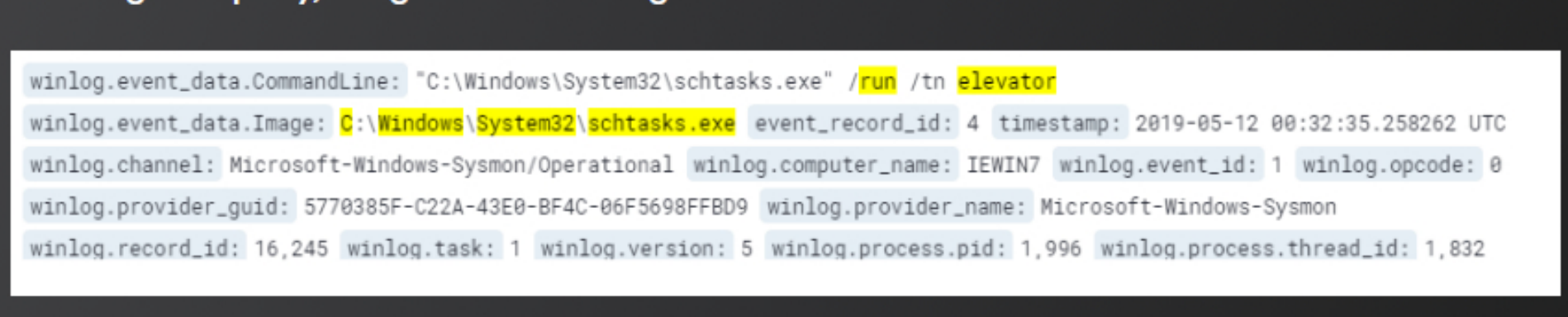


```
winlog.event_data.TargetFilename: C:\Windows\System32\Tasks\elevator event_record_id: 3 timestamp: 2019-05-12 00:32:35.258262
UTC winlog.channel: Microsoft-Windows-Sysmon/Operational winlog.computer_name: IEWIN7 winlog.event_id: 11 winlog.opcode: 0
winlog.provider_guid: 5770385F-C22A-43E0-BF4C-06F5698FFBD9 winlog.provider_name: Microsoft-Windows-Sysmon
winlog.record_id: 16,244 winlog.task: 11 winlog.version: 2 winlog.process.pid: 1,996 winlog.process.thread_id: 1,832
winlog.event_data.CreationUtcTime: 2019-05-12 00:32:30.086 winlog.event_data.Image: C:\Windows\system32\svchost.exe
```

Now that the task was created, we can identify if it was executed by modifying our first query but this time, we are looking at the command line argument that contains the task name "elevator" and the string "run" (which indicates task execution). We create the following query:

```
winlog.event_id:1 AND winlog.event_data.Image:"C:\Windows\system32\schtasks.exe" AND winlog.e
```

Running the query, we get the following match:



```
winlog.event_data.CommandLine: "C:\Windows\System32\schtasks.exe" /run /tn elevator
winlog.event_data.Image: C:\Windows\System32\schtasks.exe event_record_id: 4 timestamp: 2019-05-12 00:32:35.258262 UTC
winlog.channel: Microsoft-Windows-Sysmon/Operational winlog.computer_name: IEWIN7 winlog.event_id: 1 winlog.opcode: 0
winlog.provider_guid: 5770385F-C22A-43E0-BF4C-06F5698FFBD9 winlog.provider_name: Microsoft-Windows-Sysmon
winlog.record_id: 16,245 winlog.task: 1 winlog.version: 5 winlog.process.pid: 1,996 winlog.process.thread_id: 1,832
```

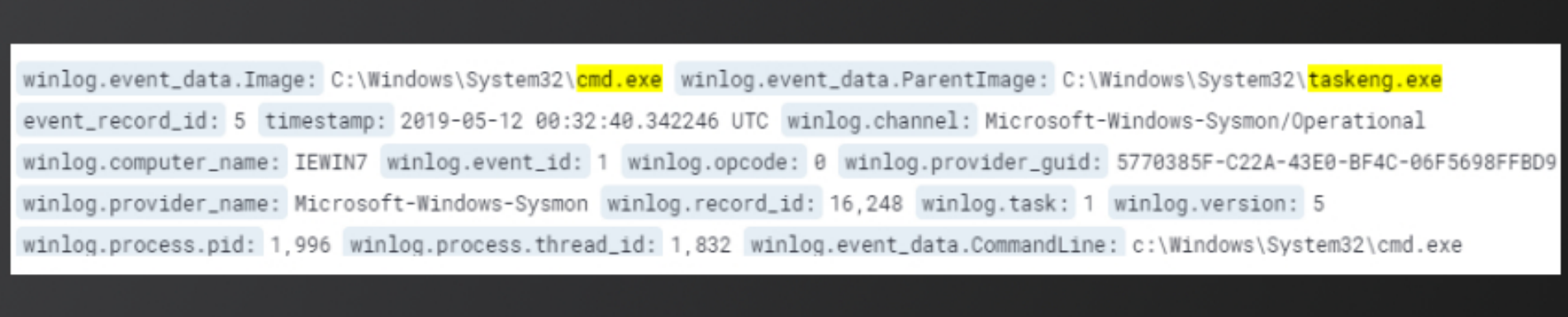
Indeed the task was executed. If we follow the documentation provided on the blog post hinted by the task, we would be interested to see if the task has spawned any of the following as the SYSTEM user:

- wscript.exe
- rundll32.exe
- cscript.exe
- regsvr32.exe
- cmd.exe
- wmic.exe
- powershell.exe

Taking into consideration that the environment is based on Windows 7 workstations and as stated by T1053, "taskeng.exe" will be the executing process starting one of those listed above. Taking this into consideration, we construct the following query:

```
winlog.event_id:1 AND winlog.event_data.ParentImage:"taskeng.exe" AND winlog.event_data.Image
```

When executed, we get the following output:



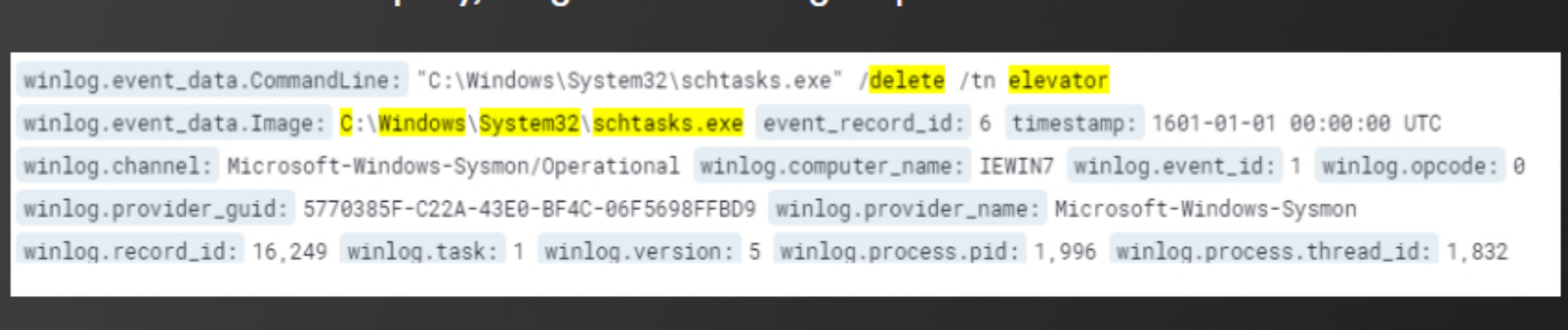
```
winlog.event_data.Image: C:\Windows\System32\cmd.exe winlog.event_data.ParentImage: C:\Windows\System32\taskeng.exe
event_record_id: 5 timestamp: 2019-05-12 00:32:40.342246 UTC winlog.channel: Microsoft-Windows-Sysmon/Operational
winlog.computer_name: IEWIN7 winlog.event_id: 1 winlog.opcode: 0 winlog.provider_guid: 5770385F-C22A-43E0-BF4C-06F5698FFBD9
winlog.provider_name: Microsoft-Windows-Sysmon winlog.record_id: 16,248 winlog.task: 1 winlog.version: 5
winlog.process.pid: 1,996 winlog.process.thread_id: 1,832 winlog.event_data.CommandLine: c:\Windows\System32\cmd.exe
```

Command prompt was started several milliseconds after the scheduled task was executed. If we expand the fields, we will also note that it was executed as "NT AUTHORITY\SYSTEM".

Finally, we can search for traces of task deletion in a similar fashion as we searched whether

```
winlog.event_id:1 AND winlog.event_data.Image:"C:\Windows\system32\schtasks.exe" AND winlog.e
```

When we execute the query, we get the following output:



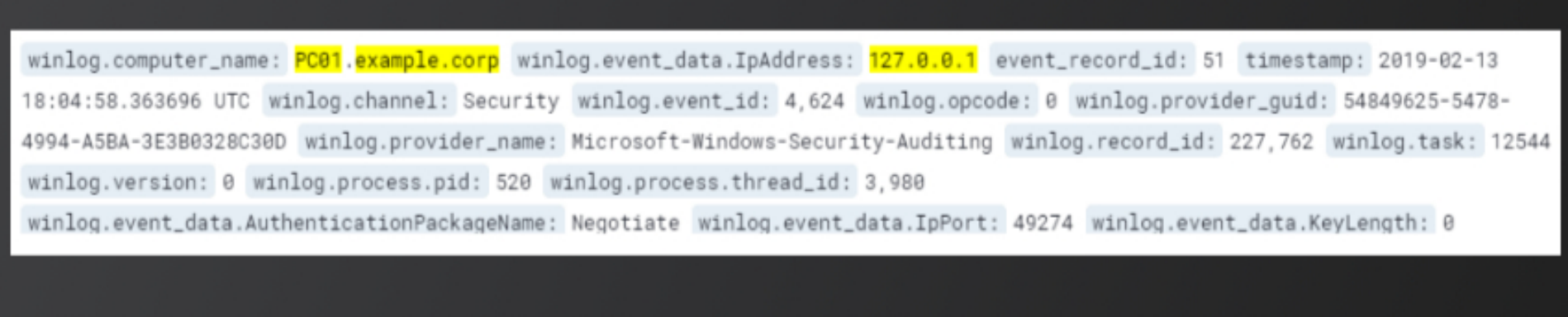
```
winlog.event_data.CommandLine: "C:\Windows\System32\schtasks.exe" /delete /tn elevator
winlog.event_data.Image: C:\Windows\System32\schtasks.exe event_record_id: 6 timestamp: 1601-01-01 00:00:00 UTC
winlog.channel: Microsoft-Windows-Sysmon/Operational winlog.computer_name: IEWIN7 winlog.event_id: 1 winlog.opcode: 0
winlog.provider_guid: 5770385F-C22A-43E0-BF4C-06F5698FFBD9 winlog.provider_name: Microsoft-Windows-Sysmon
winlog.record_id: 16,249 winlog.task: 1 winlog.version: 5 winlog.process.pid: 1,996 winlog.process.thread_id: 1,832
```

Task 4. Hunt for RDP over a Reverse SSH Tunnel

By reviewing the hinted post, to discover this activity, we should be looking at Event ID 4624 (Successful logon) with Logon type 10 (indicating RDP). Moreover, we should ensure that the network source address points to the loop IP address (127.0.0.1). Converting this into ELK query, and taking into account the computer name given in the task, we get the following:

```
winlog.event_id:4624 AND winlog.event_data.LogonType:10 AND winlog.event_data.IpAddress: 127.
```

When executed, we get the following output:



```
winlog.computer_name: PC01.example.corp winlog.event_data.IpAddress: 127.0.0.1 event_record_id: 51 timestamp: 2019-02-13
18:04:58.363696 UTC winlog.channel: Security winlog.event_id: 4,624 winlog.opcode: 0 winlog.provider_guid: 54849625-5478-
4994-A5BA-3E3B0328C30D winlog.provider_name: Microsoft-Windows-Security-Auditing winlog.record_id: 227,762 winlog.task: 12544
winlog.version: 0 winlog.process.pid: 520 winlog.process.thread_id: 3,980
winlog.event_data.AuthenticationPackageName: Negotiate winlog.event_data.IpPort: 49274 winlog.event_data.KeyLength: 0
```

Here we see that we have a match and have therefore successfully identified the activity our hunt is focused on.

KEY TAKEWAYS

1. There are numerous ways for an adversary to achieve their goal. Being familiar with as many as possible attacks can aid hunters in creating better detection rules performing successful hunts.
2. Hunting playgrounds are a great source of different attacks, helping hunters in increasing their abilities.